

EXPLAINABLE AND BIAS-AWARE MULTI-LINGUAL SENTIMENT ANALYSIS WEB PLATFORM FOR INDIAN CODE-SWITCHED TEXT

A mini-project system design report

Submitted by

Guided by

FAUSTENA S (2548313)

Dr. GAYATHRY S WARRIER

VISHWA KARTHICK S (2548342)

in partial fulfillment of the requirement for the award of the degree of
MASTER OF DATA SCIENCE



Department of Computer Science
School of Sciences
CHRIST (Deemed to be University)
Bangalore Yeshwanthpur Campus
JUNE 2026

1. System Architecture

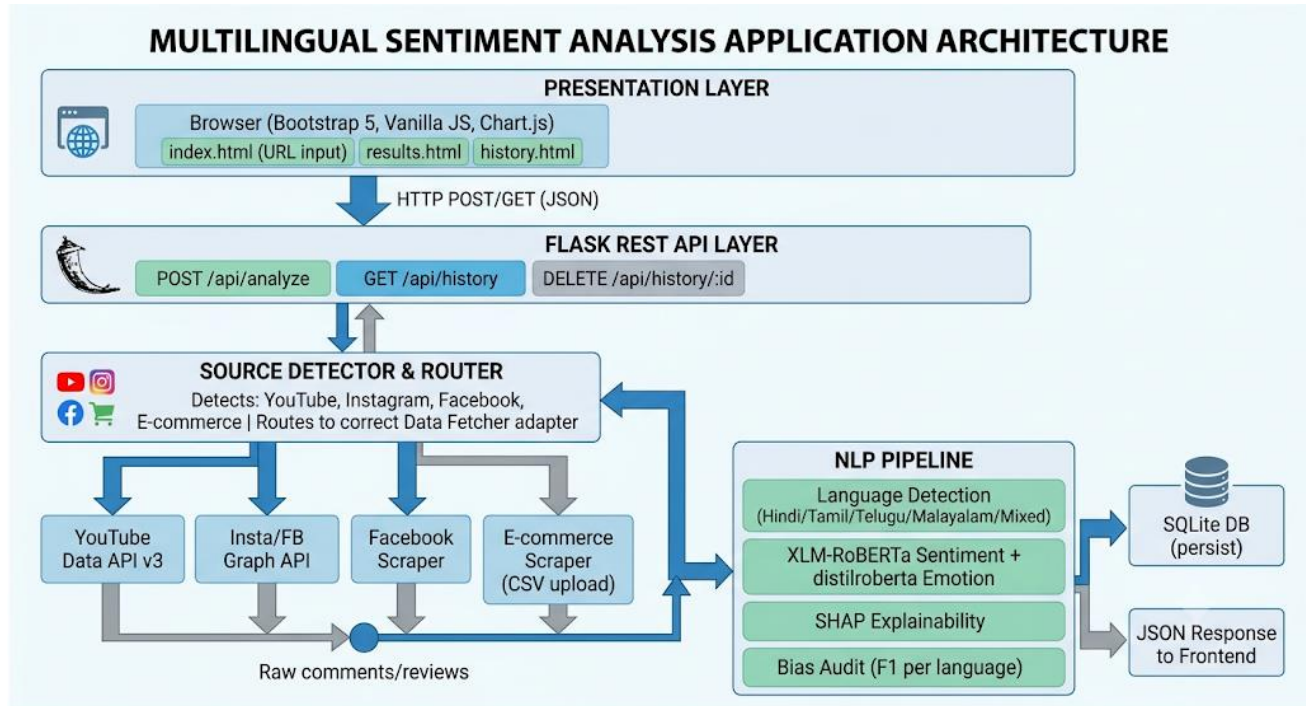


Figure 1: Overall system architecture of LinguaLens

LinguaLens follows a client-server architecture with a clear separation between the presentation layer, the API layer, and the analysis pipeline. The Frontend (HTML, Bootstrap 5, vanilla JavaScript, Chart.js) accepts a link from the user and communicates with the Flask REST API over HTTP using JSON.

The API layer exposes two primary endpoints: `/api/analyze`, which accepts a URL and returns a complete sentiment report, and `/api/history`, which returns previously stored analyses. When a request reaches `/api/analyze`, the Source Detector & Router identifies whether the link belongs to YouTube, Instagram, Facebook, or an e-commerce product page, and forwards the request to the matching data-fetching adapter — YouTube Data API v3, Instagram/Facebook Graph API, or the e-commerce scraper (for platforms like Amazon and Flipkart, users may also upload a CSV directly).

The raw comments or reviews returned by the adapters are passed to the NLP Pipeline, which performs language detection (Hindi, Tamil, Telugu, Malayalam, English, and their code-switched forms such as Hinglish and Tanglish), sentiment and emotion classification using a fine-tuned XLM-RoBERTa model, and SHAP-based explainability to surface the words that most influenced each prediction. A Bias Audit module then computes per-language F1 performance metrics so that the system's accuracy across all four Indian language groups can be monitored.

Finally, results are persisted in an SQLite database (for history and repeat viewing) and returned as a JSON response to the frontend, which renders the sentiment dashboard using Chart.js.

2. Module Design

Module	Description	Input	Output
Source Detector & Router	Identifies the platform (YouTube / Instagram / Facebook / E-commerce) from the pasted URL and routes the request to the correct fetcher.	Raw URL string	Source type + validated URL
Data Fetcher (per platform)	Retrieves comments or reviews from the identified platform using its API or a scraper. E-commerce users may upload a CSV directly.	Validated URL or CSV file	List of raw comment/review text with metadata
Language Detection	Classifies each comment into Hindi, Tamil, Telugu, Malayalam, English, or a code-switched form (Hinglish, Tanglish, Tenglish, Manglish) using langdetect and Unicode block detection.	Raw comment text	Comment tagged with a language label
Sentiment & Emotion Analysis	Runs the fine-tuned XLM-RoBERTa model to classify sentiment (positive/negative/neutral) and the distilroberta-base model to detect emotion (joy/anger/sadness/fear/surprise).	Language-tagged comment	Sentiment label, emotion label, confidence score
Explainability (SHAP)	Computes word-level contribution scores using SHapley Additive exPlanations to explain why a comment was classified as positive or negative. Applied to the top 10 most influential comments.	Comment + model prediction	Ranked list of influential words with attribution scores
Bias Audit	Computes F1-score per language group (Tamil, Hindi, Telugu, Malayalam) to detect whether the model performs unevenly across Indian languages. Flags gaps exceeding 10%.	All classified comments grouped by language	Per-language F1 scores and fairness warning flags
Flask REST API	Exposes /api/analyze and /api/history endpoints; orchestrates the full pipeline and formats the JSON response returned to the frontend.	HTTP request (URL / query params)	JSON response with all results
Database (SQLite)	Persists each analysis, its aggregate sentiment breakdown, and individual comment-level records across three relational tables.	Analysis results object	Stored rows in analyses, sentiment_details, comments tables
Dashboard & Visualisation	Renders sentiment distribution (donut chart), emotion profile (radar chart), and language mix (bar chart) plus top comment cards and SHAP word highlights.	JSON analysis response	Rendered dashboard with interactive Chart.js visualisations
History Management	Lists, searches, filters by platform or sentiment, paginates, and deletes past analyses. Interfaces with the SQLite database via GET and DELETE endpoints.	Paginated query + optional filters	Table of past analyses with delete action per row

3. System Workflow

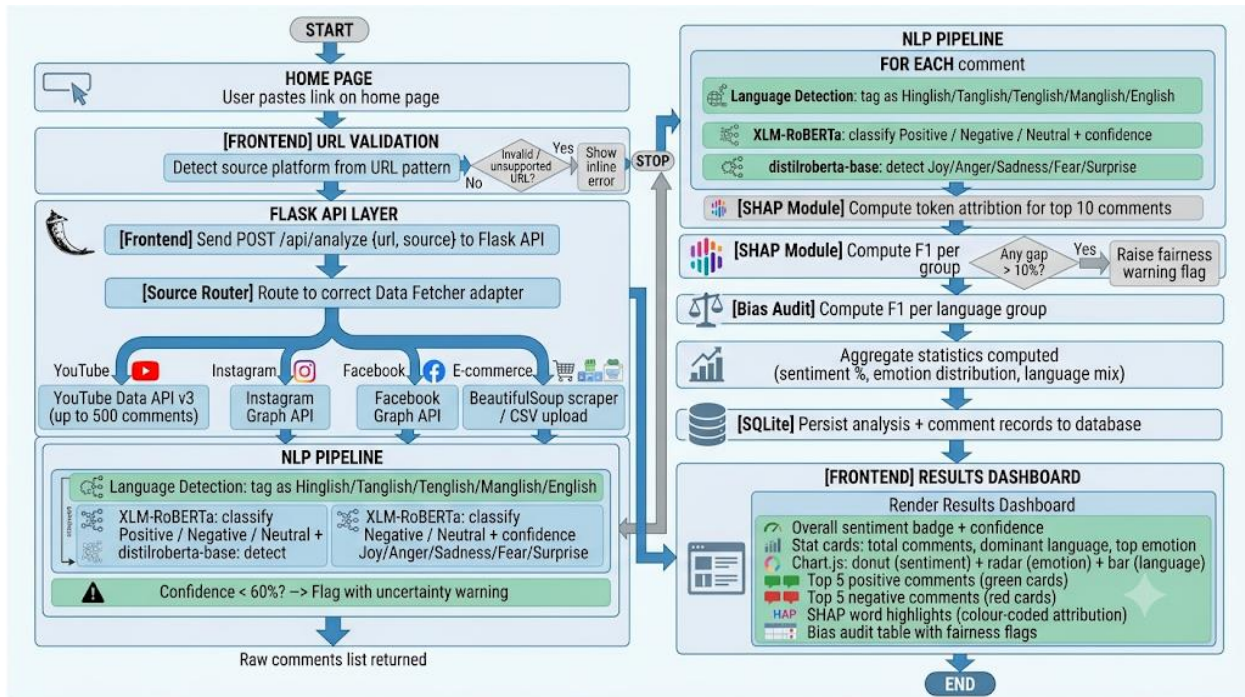


Figure 2: End-to-end workflow from URL submission to dashboard rendering

The workflow begins when the user pastes a link into the home page. The frontend first detects the source platform from the URL pattern and performs a basic validity check; an unsupported or malformed link results in an inline error message without contacting the server. Once validated, the URL and detected source are sent to the backend.

The backend fetches the raw comments or reviews using the adapter that matches the detected source, then runs each comment through the language detector, the XLM-RoBERTa sentiment and emotion model, and the SHAP explainability step. Aggregate statistics and the bias audit are computed once all comments have been classified. The complete result set is written to the SQLite database and returned to the frontend as a single JSON object, which is used to render the sentiment dashboard — including charts, top comment cards, SHAP word highlights, and the bias audit table.

4. Design Artefacts

4.1 Entity Relationship Diagram (ERD)

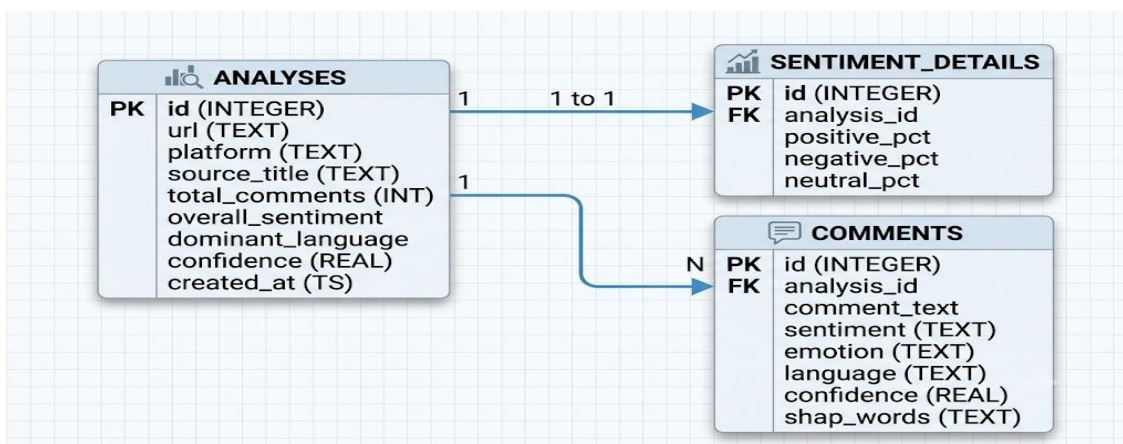


Figure 3: ERD showing analyses, sentiment_details, and comments tables

Each analysis is stored once in the analyses table, capturing the source platform, the original URL, content metadata, and the overall sentiment and dominant language summary. The sentiment_details table holds a one-to-one breakdown of positive, negative, and neutral percentages for that analysis. The comments table stores a one-to-many set of individual comment-level records including text, sentiment, emotion, language, confidence, and SHAP word attributions, linked back to analyses through the analysis_id foreign key.

4.2 Data Flow Diagram (DFD) — Level 1

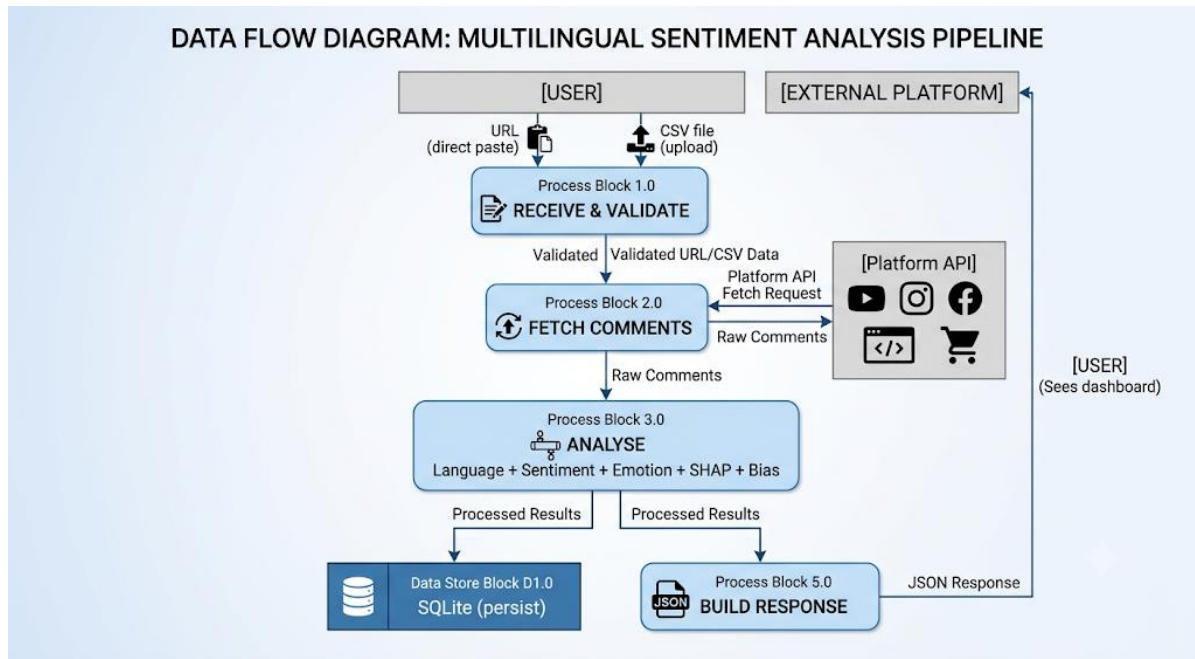


Figure 4: Level-1 DFD showing processes, external entities, and the data store

The DFD traces how data moves through the system: the User provides a URL (Process 1.0), which triggers a fetch against the relevant External Platform (Process 2.0). Fetched comments flow into language detection, sentiment classification, emotion detection, SHAP explainability, and bias auditing (Process 3.0). Process 5.0 assembles the final dashboard payload, writing a persistent copy to the data store (D1) and returning the JSON response to the User.

4.3 Algorithm / Pseudocode — Core Analysis Pipeline

ALGORITHM AnalyzeLink(url)

```

1. source ← DetectSource(url)    // youtube | instagram | facebook |
ecommerce
2. IF source is NULL THEN RETURN error("Unsupported or invalid URL")
3. rawComments ← FetchComments(source, url)    // via API or scraper
4. IF rawComments is EMPTY THEN RETURN error("No comments found")
5. classifiedComments ← []
6. FOR EACH comment IN rawComments:
    language ← DetectLanguage(comment.text)
    // Uses Unicode block ranges + langdetect
    // Returns: Hinglish | Tanglish | Tenglish | Manglish | English

    (sentiment, confidence) ← XLM_R_Classify(comment.text)
  
```



```

// Returns: Positive | Negative | Neutral, score 0.0–1.0

IF confidence < 0.60 THEN flag comment as UNCERTAIN

emotion ← distilroberta_Classify(comment.text)
// Returns: Joy | Anger | Sadness | Fear | Surprise

APPEND {comment, language, sentiment, emotion, confidence}
TO classifiedComments

7. topComments ← SELECT top 10 by |confidence| FROM classifiedComments
   shapWords ← SHAP_Explain(topComments)
   // Returns: [{word, attribution_score, sentiment_direction}]

8. stats ← AggregateStatistics(classifiedComments)
   // Computes: sentiment %, emotion distribution, language mix

9. biasReport ← ComputeF1PerLanguage(classifiedComments)
   // Computes F1 for each: Tamil | Hindi | Telugu | Malayalam
   IF MAX(F1) - MIN(F1) > 0.10 THEN biasReport.flag ← TRUE

10. SaveToDatabase(source, url, stats, biasReport, classifiedComments)

11. RETURN BuildJSONResponse(stats, biasReport, shapWords,
classifiedComments)

```

The algorithm processes each comment independently through language detection, sentiment classification, emotion detection, and confidence thresholding before computing aggregate statistics, SHAP attributions, and the bias audit in a single pass. The SQLite write occurs once at the end to avoid partial saves.

4.4 Data Dictionary

Field	Table	Type	Constraint	Description
id	analyses	INTEGER	PK AUTO	Unique identifier for each analysis session
url	analyses	TEXT	NOT NULL	Original YouTube URL, social media URL, or CSV filename
platform	analyses	TEXT	NOT NULL	Source platform: YouTube, Instagram, Facebook, E-commerce
source_title	analyses	TEXT		Title of the video, product, or post being analysed
total_comments	analyses	INTEGER		Total number of comments processed in this analysis
overall_sentiment	analyses	TEXT		Aggregate verdict: Positive, Negative, or Neutral
dominant_language	analyses	TEXT		Most frequent language pair detected across all comments
confidence	analyses	REAL		Average model confidence score across all comments (0.0–1.0)
created_at	analyses	TIMESTAMP	DEFAULT NOW	Timestamp when the analysis was stored

analysis_id	sentiment_details	INTEGER	FK analyses →	Foreign key linking to the parent analysis record
positive_pct	sentiment_details	REAL		Percentage of comments classified as positive
negative_pct	sentiment_details	REAL		Percentage of comments classified as negative
neutral_pct	sentiment_details	REAL		Percentage of comments classified as neutral
analysis_id	comments	INTEGER	FK analyses →	Foreign key linking comment to parent analysis
comment_text	comments	TEXT	NOT NULL	Original raw comment or review string
sentiment	comments	TEXT		Per-comment sentiment: Positive, Negative, or Neutral
emotion	comments	TEXT		Detected emotion: Joy, Anger, Sadness, Fear, or Surprise
language	comments	TEXT		Detected language pair: Tanglish, Hinglish, Tenglish, Manglish, English
confidence	comments	REAL		Per-comment model confidence score (0.0–1.0)
shap_words	comments	TEXT		JSON string of top SHAP word attributions for this comment

5. User Interface Design

5.1 Home Page — Link Input

LinguaLens

Understand What the World Really Thinks

Paste a YouTube / Instagram / Facebook / product link, or upload a review CSV file

[Paste link here...] Upload CSV Analyse

YouTube Instagram Facebook E-commerce

Supports Hinglish Supports Tanglish Supports Tenglish Supports Manglish

Footer — Learn More: About LinguaLens | Documentation | Contact Us | GitHub | User Privacy Policy

Figure 5: Home page where the user pastes a link and the source is auto-detected

5.2 Results Dashboard



Figure 6: Results dashboard showing sentiment, emotion, and language charts

5.3 Analysis History

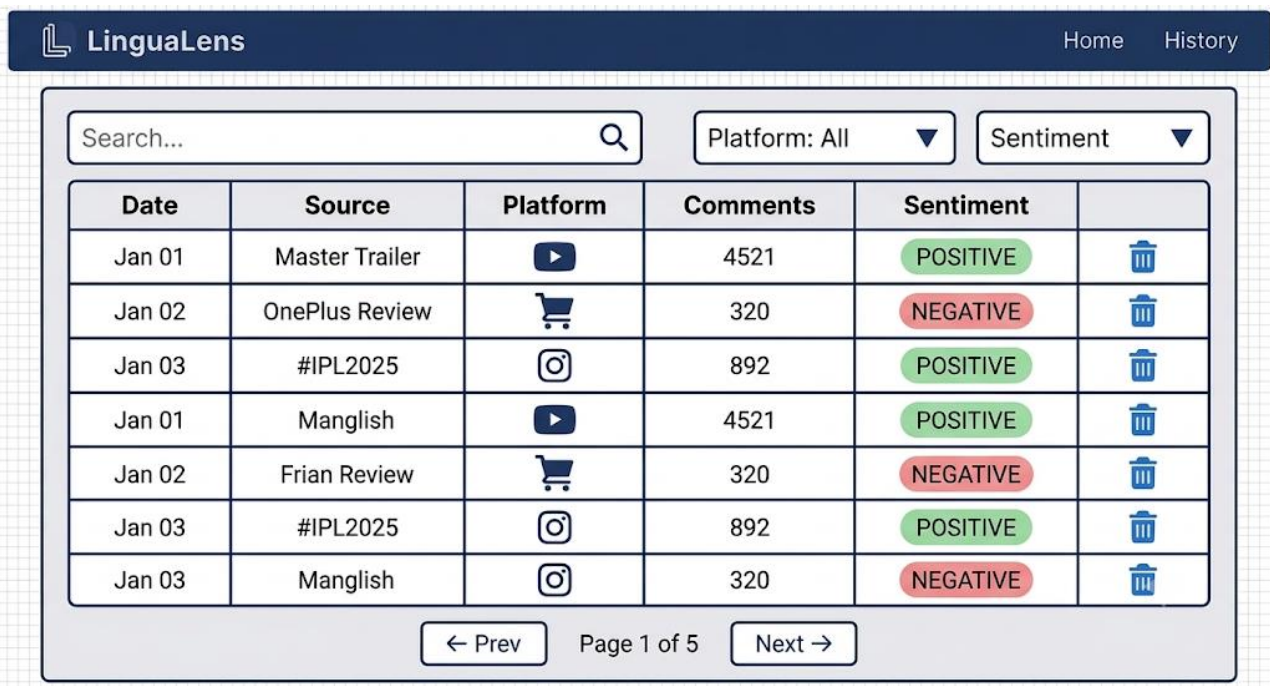


Figure 7: Wireframe of the history page with search, source filter, and pagination

Student Declaration

We hereby declare that the proposed project work is original and will be carried out under the guidance of the faculty mentor.

Student Signature(s):

Date: 29- 07- 2026

Faculty Guide Signature: